

2024-10-31-IntLinProg

Generated by Scribe.AI

December 31, 2024

Slide 1

Content

Integer Programming

$$\max \quad \xi(x) = c^T x$$

$$\text{s.t.} \quad Ax \leq b$$

$$x \geq 0, \quad x_j \in \mathbb{Z}$$

Description

Slide 1 of 23 introduces the concept of Integer Programming. The slide presents a standard integer programming problem formulation. The objective is to maximize a linear function $\xi(x) = c^T x$, where c is a vector of coefficients and x is a vector of variables. The constraints are given by $Ax \leq b$, where A is a matrix and b is a vector. Additionally, the non-negativity constraint $x \geq 0$ is imposed, and crucially, the integrality constraint $x_j \in \mathbb{Z}$ is specified, meaning that all components of the solution vector x must be integers. The "s.t." abbreviation stands for "subject to". The context of the course (Linear Programming) is important because this slide sets the stage for discussing a problem type that is a generalization of linear programming, where the variables are restricted to integer values, making the problem significantly more complex to solve.

Figures



(a) Handwritten notes showing the formulation of an integer programming problem. The objective function is to maximize $\xi(x) = c^T x$, subject to the constraints $Ax \leq b$, $x \geq 0$, and $x_j \in \mathbb{Z}$ for all j .

Slide 2

Content

Integer Programming

$$\max \quad \xi(x) = c^T x$$

$$\text{s.t.} \quad Ax \leq b$$

$$x \geq 0, \quad x_j \in \mathbb{Z}$$

Rk: Solving the problem without the int. constraints

and then round up/down the solution does

not work because

(1) rounded solution might not be feasible

(2) true opt. int solution might be far from

the rounded solution.

Description

Slide 2 of 23 discusses integer programming and the limitations of a naive approach to solving it. The slide begins by formally defining an integer programming problem: maximize $\xi(x) = c^T x$ subject to $Ax \leq b$, $x \geq 0$, and $x_j \in \mathbb{Z}$ for all j . The notation is standard for linear programming, with c representing the objective function coefficients, A the constraint matrix, b the constraint vector, and x the vector of decision variables. The crucial difference from linear programming is the integrality constraint $x_j \in \mathbb{Z}$, indicating that the variables must take integer values. The slide then explains why simply solving the relaxed problem (ignoring the integrality constraints) and rounding the solution is not a reliable method. Two reasons are given: (1) the rounded solution might not be feasible (i.e., it might violate the constraints $Ax \leq b$), and (2) the optimal integer solution might be far from the rounded solution, leading to a suboptimal result. The term "relaxed problem" refers to the linear programming problem obtained by removing the integrality constraints. The context of the course (Linear Programming) is essential because this slide highlights the key differences and challenges of integer programming compared to its linear counterpart.

Figures

Integer Programming

$$\max \quad \xi(x) =$$

$$\text{s.t.} \quad Ax \leq$$

$$x \geq 0$$

(a) Handwritten notes describing integer programming and the limitations of rounding solutions from the relaxed problem.

Slide 3

Content

IV] P. 397

maximize $17x_1 + 12x_2$

subject to $10x_1 + 7x_2 \leq 40$

$x_1 + x_2 \leq 5$

$x_1, x_2 \geq 0$

x_1, x_2 integers.

Description

Slide 3 of 23 presents a specific integer programming problem taken from a textbook (indicated by "IV] P. 397"). The problem is to maximize the linear objective function $17x_1 + 12x_2$ subject to the constraints $10x_1 + 7x_2 \leq 40$, $x_1 + x_2 \leq 5$, and $x_1, x_2 \geq 0$. The key feature, highlighted by underlining, is that the decision variables x_1 and x_2 are restricted to integer values. This makes it an integer programming problem, which is more complex to solve than a standard linear programming problem (where the variables can take on any real value). The context of the course (Linear Programming) is crucial because this slide provides a concrete example of the type of problem the course is addressing, illustrating the difference between linear programming and integer programming. The reference to a page number in a textbook suggests that this problem will be used as an example in the lecture.

Figures

IV] p. 397

(a) Handwritten notes showing an integer programming problem. The objective function is to maximize $17x_1 + 12x_2$, subject to the constraints $10x_1 + 7x_2 \leq 40$, $x_1 + x_2 \leq 5$, $x_1 \geq 0$, $x_2 \geq 0$, and x_1, x_2 being integers.

Slide 4

Content

IV] P. 397

maximize $17x_1 + 12x_2$

subject to $10x_1 + 7x_2 \leq 40$

$x_1 + x_2 \leq 5$

$x_1, x_2 \geq 0$

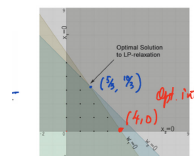
x_1, x_2 integers.

$$x_1^* = \frac{5}{3}, \quad x_2^* = \frac{10}{3}$$

Description

Slide 4 of 23 presents an integer programming problem and its graphical solution. The problem, referenced from a textbook ("IV] P. 397"), is to maximize $17x_1 + 12x_2$ subject to $10x_1 + 7x_2 \leq 40$, $x_1 + x_2 \leq 5$, $x_1, x_2 \geq 0$, and the integrality constraint $x_1, x_2 \in \mathbb{Z}$. The slide shows the optimal solution to the *relaxed* linear programming problem (i.e., ignoring the integrality constraint) as $x_1^* = \frac{5}{3}$ and $x_2^* = \frac{10}{3}$. A graph displays the feasible region, highlighting the optimal solution to the relaxed LP problem $\left(\frac{5}{3}, \frac{10}{3}\right)$ and the optimal integer solution $(4, 0)$. The dots in the graph represent feasible integer points. The text "Opt. int soln" labels the optimal integer solution. The context of the course (Linear Programming) is crucial because this slide visually demonstrates the difference between the solution of a linear program and its integer counterpart. The slide emphasizes that simply rounding the solution of the relaxed LP problem does not yield the optimal integer solution. The graph visually shows that the optimal integer solution is different from the rounded solution of the relaxed problem.

Figures



(a) Handwritten notes showing the solution to the relaxed LP problem.

(b) A graph showing the feasible region of the integer programming problem and the optimal solutions to the relaxed LP problem and the integer programming problem.

Slide 5

Content

Branch-and-Bound

maximize $17x_1 + 12x_2$

subject to $10x_1 + 7x_2 \leq 40$

$x_1 + x_2 \leq 5$

$x_1, x_2 \geq 0$

x_1, x_2 integers.

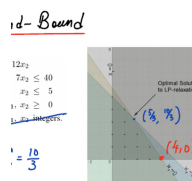
$x_1^* = \frac{5}{3}, \quad x_2^* = \frac{10}{3}$

$x_1 \leq 1 \quad x_1 \geq 2 \leftarrow 2$ sub-problems, with new constraints

Description

Slide 5 of 23 introduces the branch-and-bound method for solving integer programming problems. The problem, previously introduced, is to maximize $17x_1 + 12x_2$ subject to $10x_1 + 7x_2 \leq 40$, $x_1 + x_2 \leq 5$, $x_1, x_2 \geq 0$, and x_1, x_2 being integers. The optimal solution to the relaxed linear programming problem (ignoring the integer constraints) is given as $x_1^* = \frac{5}{3}$ and $x_2^* = \frac{10}{3}$. The core of the slide is the illustration of the branching step in the branch-and-bound algorithm. Because the optimal solution to the relaxed problem is not integer, the algorithm branches into two subproblems by adding new constraints: $x_1 \leq 1$ and $x_1 \geq 2$. These constraints partition the feasible region, creating two smaller subproblems that will be solved recursively. The graph visually represents this branching process, showing the feasible region, the optimal solution to the relaxed LP problem, and the optimal integer solution. The arrows indicate how the branching process creates two new subproblems with additional constraints. The context of the course (Linear Programming) is crucial because this slide explains a method to solve integer programming problems, which are more complex than linear programming problems, by leveraging the solution of the relaxed linear programming problem.

Figures



(a) A graph showing the feasible region of the integer programming problem, the optimal solution to the relaxed LP problem, and the optimal integer solution. Arrows point from the relaxed solution to two new constraints, $x_1 \leq 1$ and $x_1 \geq 2$.

Slide 6

Content

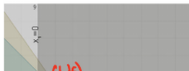
Branch-and-Bound

Description

Slide 6 of 23 illustrates the branch-and-bound algorithm applied to an integer programming problem. The left side shows a graphical representation of the feasible region, partitioned into subregions by branching on the variable x_1 . Points (1, 4) and (2, 2.86) are highlighted, likely representing feasible integer solutions or intermediate points in the algorithm. The right side shows a partial branch-and-bound tree. The root node represents the solution to the relaxed linear programming problem, with $x_1 = \frac{5}{3}$ and $x_2 = \frac{10}{3}$, yielding an objective function value of approximately 68.33. Branching occurs on x_1 , creating two subproblems: $x_1 \leq 1$ and $x_1 \geq 2$. The resulting nodes show the objective function values for these subproblems, approximately 65 and 68.29. The figure numbers (23.6 and 23.7) indicate that these figures are referenced from a textbook or other source material. The context of the course (Linear Programming) is crucial because this slide visually demonstrates a key algorithm for solving integer programming problems, a topic directly related to linear programming.

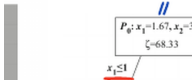
Figures

Branch-and-Bound



(a) A graph showing the feasible region of an integer programming problem. Points (1,4) and (2, 2.86) are marked. The feasible region is divided into subregions by the lines $x_1 = 1$ and $x_1 = 2$. The labels P_1 and P_2 are present.

und



(b) A branch-and-bound tree. The root node represents the relaxed linear programming problem with solution $x_1 = \frac{5}{3}, x_2 = \frac{10}{3}$ and objective function value ≈ 68.33 . The branches represent the constraints $x_1 \leq 1$ and $x_1 \geq 2$. The nodes resulting from the branching represent subproblems with these additional constraints. The objective function values for these subproblems are given as ≈ 65 and ≈ 68.29 .

Slide 7

Content

Branch-and-Bound

Description

Slide 7 of 23 visually depicts the branch-and-bound algorithm's progress. The left figure (Figure 23.8) shows a 3D graphical representation of the feasible region of an integer programming problem. The feasible region is divided into subregions (P_1 , P_2 , P_3), illustrating the refinement process within the branch-and-bound algorithm. A point (2, 2) is highlighted, likely representing a feasible integer solution. The right figure (Figure 23.9) displays the corresponding branch-and-bound tree. Each node represents a subproblem created by adding constraints on the variables (x_1 and x_2). The objective function value for each subproblem is shown, along with the values of x_1 and x_2 at the solution of the relaxed LP problem for that subproblem. A red line highlights the best integer solution found so far in the search tree. The figures demonstrate the interplay between the geometric representation of the feasible region and the tree structure of the branch-and-bound algorithm. The context of the course (Linear Programming) is crucial because this slide visually demonstrates the branch-and-bound algorithm, a key technique for solving integer programming problems, which are a direct extension of linear programming problems.

Figures

Branch-and-Bound



(a) A 3D graph showing the feasible region of an integer programming problem. The feasible region is partitioned into subregions P_1 , P_2 , and P_3 . Point $(2, 2)$ is marked with a red dot. The axes are labeled x_1 and x_2 . The caption is "FIGURE 23.8. The refinement of P_2 to P_3 ."

and



(b) A branch-and-bound tree. The root node represents the relaxed linear programming problem with solution $x_1 \approx 1.67$, $x_2 \approx 3.33$ and objective function value ≈ 68.33 . The branches represent the constraints $x_1 \leq 1$ and $x_1 \geq 2$. Further branching occurs on x_2 . The nodes represent subproblems with these additional constraints. The objective function values for these subproblems are given. A red line highlights the best integer solution found so far. The caption is "FIGURE 23.9. The enumeration tree after solving P_3 ."

Slide 8

Content

Branch-and-Bound

Description

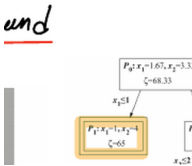
Slide 8 of 23 continues the illustration of the branch-and-bound algorithm. The left figure (Figure 23.10) shows a 3D visualization of the feasible region being partitioned into subregions (P_1 , P_2 , P_3 , P_4) during the branching process. Points (2,2) and (3, 1.43) are highlighted as potential integer solutions. The right figure (Figure 23.11) displays the corresponding branch-and-bound tree, showing the progression of the algorithm. Each node represents a subproblem with added constraints on x_1 , x_2 , and x_3 . The objective function value for each subproblem is shown, along with the solution to the relaxed LP problem for that subproblem. A double box around node P_4 signifies that it is a leaf node (no further branching is needed). The figures demonstrate the iterative refinement of the feasible region and the exploration of the solution space through the branch-and-bound tree. The context of the course (Linear Programming) is crucial because this slide visually demonstrates the branch-and-bound algorithm's progress in solving an integer programming problem, showing how the algorithm explores the solution space and refines the feasible region until an optimal integer solution is found.

Figures

Branch-and-Bou



(a) A 3D graph showing the feasible region of an integer programming problem. The feasible region is partitioned into subregions P_1 , P_2 , P_3 , and P_4 . Points $(2,2)$ and $(3, 1.43)$ are marked with red dots. The axes are labeled x_1 and x_2 . The caption is "FIGURE 23.10. The refinement of P_2 to P_3 and P_4 . Note that P_4 is just a short vertical line segment."



(b) A branch-and-bound tree. The root node represents the relaxed linear programming problem with solution $x_1 \approx 1.67$, $x_2 \approx 3.33$ and objective function value ≈ 68.33 . The branches represent the constraints $x_1 \leq 1$ and $x_1 \geq 2$. Further branching occurs on x_2 and x_3 . The nodes represent subproblems with these additional constraints. The objective function values for these subproblems are given. A double box around P_4 indicates that it is a leaf node. The caption is "FIGURE 23.11. The enumeration tree after solving P_3 . The double box around P_4 indicates that it is a leaf in the tree."

Slide 9

Content

Branch-and-Bound

Description

Slide 9 of 23 shows the branch-and-bound algorithm's progress. The left figure (Figure 23.12) displays a 2D graphical representation of the feasible region, partitioned into subregions (P_1 , P_4 , P_5 , P_6) during the branching process. The point (3, 3) is highlighted, likely representing a feasible integer solution. The right figure (Figure 23.14) shows the corresponding branch-and-bound tree. Each node represents a subproblem created by adding constraints on the variables (x_1 , x_2 , x_3). The objective function value for each subproblem is shown, along with the solution to the relaxed LP problem for that subproblem. The nodes P_3 , P_4 , P_5 , and P_6 are highlighted, indicating the subproblems explored. The figures demonstrate the iterative refinement of the feasible region and the exploration of the solution space through the branch-and-bound tree. The context of the course (Linear Programming) is crucial because this slide visually demonstrates the branch-and-bound algorithm's progress in solving an integer programming problem, showing how the algorithm explores the solution space and refines the feasible region until an optimal integer solution is found. The figures show the algorithm has progressed beyond the previous slide, exploring further branches of the tree.

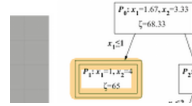
Figures

Branch-and-Bou



(a) A 2D graph showing the feasible region of an integer programming problem. The feasible region is a polygon bounded by the lines $x_2 = 0$, $x_1 = 0$, and $x_1 + x_2 = 6$. The region is partitioned into subregions P_1 , P_4 , P_5 , and P_6 . Point $(3,3)$ is marked with a red dot. The axes are labeled x_1 and x_2 . The caption is "FIGURE 23.12. The refinement of P_5 to P_6 ."

und



(b) A branch-and-bound tree. The root node represents the relaxed linear programming problem with solution $x_1 \approx 1.67$, $x_2 \approx 3.33$ and objective function value ≈ 68.33 . The branches represent the constraints $x_1 \leq 1$ and $x_1 \geq 2$. Further branching occurs on x_2 and x_3 . The nodes represent subproblems with these additional constraints. The objective function values for these subproblems are given. The nodes P_3 , P_4 , P_5 , and P_6 are highlighted. The caption is "FIGURE 23.14. The enumeration tree after solving P_5 , P_7 , and P_8 ."

Slide 10

Content

Branch-and-Bound

Description

Slide 10 of 23 illustrates the complete enumeration tree generated by the branch-and-bound algorithm for solving an integer programming problem. The left figure (Figure 23.13) shows a 2D graphical representation of the feasible region, partitioned into subregions (P_6 , P_7 , P_8) during the branching process. Points $(3, 3)$, $(4, 0)$, and $(1, 5)$ are highlighted as integer solutions within the partitioned regions. The right figure (Figure 23.15) displays the complete branch-and-bound tree. Each node represents a subproblem with added constraints on x_1 and x_2 . The objective function value (z) for each subproblem is shown, along with the solution to the relaxed LP problem for that subproblem. Leaf nodes P_7 and P_8 are marked as infeasible, indicating that no feasible integer solutions exist within those subregions. The figures demonstrate the complete exploration of the solution space through the branch-and-bound tree, ultimately identifying the optimal integer solution. The context of the course (Linear Programming) is crucial because this slide visually demonstrates the complete process of the branch-and-bound algorithm in solving an integer programming problem, showing how the algorithm systematically explores the solution space and refines the feasible region until an optimal integer solution is found or infeasibility is proven for all remaining subproblems.

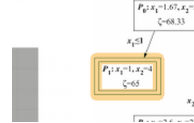
Figures

Branch-and-Bound



(a) A 2D graph showing the feasible region of an integer programming problem. The feasible region is a polygon bounded by the lines $x_2 = 0$, $x_1 = 0$, and $x_1 + x_2 = 6$. The region is partitioned into subregions P_6 , P_7 , and P_8 . Points (3, 3), (4, 0), and (1, 5) are marked with red dots. The axes are labeled x_1 and x_2 . The caption is "FIGURE 23.13. The refinement of P_6 to P_7 and P_8 . Note that P_8 consists of just a single point: (4, 0)."

and



(b) A branch-and-bound tree. The root node represents the relaxed linear programming problem with solution $x_1 \approx 1.67$, $x_2 \approx 3.33$ and objective function value ≈ 68.33 . The branches represent the constraints $x_1 \leq 1$ and $x_1 \geq 2$. Further branching occurs on x_2 and x_3 . The nodes represent subproblems with these additional constraints. The objective function values for these subproblems are given. The nodes P_7 and P_8 are leaf nodes and are marked as infeasible. The caption is "FIGURE 23.15. The complete enumeration tree."

Slide 11

Content

Branch-and-Bound

It is better going deep - depth search

before going wide - breadth search

Description

Slide 11 of 23 discusses the advantages of depth-first search over breadth-first search in the branch-and-bound algorithm for integer programming problems. The slide begins by stating that a depth-first search ("going deep before going wide") is generally preferred. Three reasons are given. The first reason is that most integer solutions are found deep within the enumeration tree. Two advantages of finding a feasible integer solution early are then explained. First, it allows for early termination of the search if a satisfactory solution is found. Second, and more importantly, finding a feasible integer solution allows for "pruning" the tree. This pruning occurs because any node with an objective function value worse than the best feasible integer solution found so far can be eliminated from further consideration, significantly improving efficiency. The highlighted terms "integer solutions" and "feasible solution" emphasize the core concept of finding integer solutions within the feasible region of the problem. The context of linear programming is crucial because this slide explains a key algorithmic optimization technique used to solve integer programming problems, a common extension of linear programming.

Figures

(a) A circled number 1.

(b) A circled number 1.

Slide 12

Content

Branch-and-Bound

It is better going deep - depth search

before going wide - breadth search

② A second reason to favor depth-first search is the simple fact that it is very easy to code the algorithm as a recursively defined function. This may seem trite, but one should not underestimate the value of code simplicity when implementing algorithms that are otherwise quite sophisticated, such as the one we are currently describing.

Description

Slide 12 of 23 continues the discussion on depth-first search versus breadth-first search within the branch-and-bound algorithm for integer programming. The slide emphasizes that depth-first search is preferable, not only because most integer solutions are found deep in the search tree (as discussed in the previous slide), but also because it simplifies the algorithm's implementation. The main point is that a depth-first search can be easily coded as a recursively defined function. While this might seem trivial, the slide argues that code simplicity is valuable when dealing with complex algorithms like branch-and-bound. The circled "2" indicates this is the second reason given for preferring depth-first search. The context of linear programming is crucial because this slide discusses implementation details and efficiency considerations for solving integer programming problems, a common extension of linear programming.

Figures



(a) A circled number 2.

Slide 13

Content

Description

Slide 13 of 23 provides the third and most important reason to prefer depth-first search in the branch-and-bound algorithm for integer programming problems within the context of linear programming. It explains that as the search goes deeper, each subsequent linear program is a refinement of the previous one, simply adding (or refining) an upper or lower bound on a specific variable. This is advantageous because it allows for efficient pruning of the search tree. The slide uses the example of problems P_0 and P_2 . P_0 's optimal dictionary (simplex tableau) is shown, and P_2 is obtained by adding the constraint $x_1 \geq 2$. A new variable $g_1 = x_1 - 2$ is introduced, and its expression in terms of nonbasic variables (w_1, w_2) is derived from P_0 's dictionary. The underlining of "refining" and "upper/lower" emphasizes the key aspect of this refinement process. The context of linear programming is crucial because this slide explains how the branch-and-bound algorithm leverages the structure of linear programs to efficiently explore the integer feasible region.

Figures

before going

The third reason to favor depth in the observation that as one refines a linear programming problem is by refining an upper/lower bound on

(a) A simplex tableau for problem P_0 .

linear programming problem is by refining an upper/lower bound on. For example, the optimal dictionary for problem P_0 is

(b) An arrow pointing to the equation $g_1 = x_1 - 2 = -\frac{1}{3}w_1 + \frac{7}{3}w_2$.

Slide 14

Content

Branch-and-Bound

It is better going deep - depth search

before going wide - breadth search

③ Therefore, we can use the following dictionary as a starting point for the solution

of P_2 :

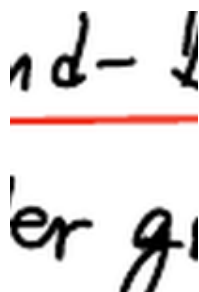
$$\begin{aligned}\zeta &= \frac{205}{3} - \frac{5}{3}w_1 - \frac{1}{3}w_2 \leftarrow \text{dual feasible} \\ x_1 &= \frac{5}{3} - \frac{1}{3}w_1 + \frac{7}{3}w_2 \\ x_2 &= \frac{10}{3} + \frac{1}{3}w_1 - \frac{10}{3}w_2 \\ g_1 &= -1 + \frac{1}{3}w_1 + \frac{7}{3}w_2 \leftarrow \text{new constraint}\end{aligned}$$

This dictionary is dual feasible but primal infeasible. Therefore, the dual simplex method is likely to find a new optimal solution in very few iterations. According to

Description

Slide 14 of 23 presents the third reason for preferring depth-first search in branch-and-bound: efficiency gains from warm starts. The slide introduces a new problem, P_2 , which is a refinement of a previous problem (likely P_1 or P_0). A simplex tableau (dictionary) for P_2 is shown. Crucially, this dictionary is *dual feasible* but *primal infeasible*. The dual feasibility means that the current solution is optimal for the dual problem. The primal infeasibility means that the current solution is not feasible for the primal problem (the original problem). The dual simplex method is then proposed as an efficient way to solve P_2 because it starts from a dual feasible solution, requiring only a few iterations to find a new optimal solution. The new constraint $g_1 = -1 + \frac{1}{3}w_1 + \frac{7}{3}w_2$ is highlighted, indicating that P_2 is obtained by adding a constraint to a previous problem. The context of linear programming is crucial because this slide demonstrates how the branch-and-bound algorithm leverages the properties of linear programs and the dual simplex method to efficiently explore the search tree.

Figures



(a) A circled number 3.

going deep
going wide

(b) A simplex tableau for problem P_2 .

wide -

- bread

(c) An arrow pointing to the equation $\zeta = \frac{205}{3} - \frac{5}{3}w_1 - \frac{1}{3}w_2$.

(d) An arrow pointing to the equation $g_1 = -1 + \frac{1}{3}w_1 + \frac{7}{3}w_2$.

Slide 15

Content

Gomory Cuts

maximize $8x_1 + 13x_2$

subject to $10x_1 - 2x_2 \leq 32$

$4x_1 + 10x_2 \leq 38$

$-2x_1 + 10x_2 \leq 29$

$x_1, x_2 \geq 0$

x_1, x_2 integers.

Description

Slide 15 of 23 introduces the Gomory cuts method for solving integer programming problems within the context of linear programming. The slide presents an integer programming problem: maximize $8x_1 + 13x_2$ subject to three linear constraints and non-negativity constraints on x_1 and x_2 , where x_1 and x_2 are integers. The note on the right clarifies that all coefficients in the constraints (A , b) and the variables (X) are integers, and importantly, that the slack variables introduced to convert inequalities to equalities are also integers. This integer restriction on all variables and coefficients is crucial for the applicability of Gomory cuts. The context of linear programming is essential because this slide introduces a method to solve integer programming problems, which are a more complex extension of linear programming problems.

Figures

Note:

A, b, X

(a) Note: A , b , X are integers and so are slacks.

Slide 16

Content

Gomory Cuts

maximize $8x_1 + 13x_2$

subject to $10x_1 - 2x_2 \leq 32$

$4x_1 + 10x_2 \leq 38$

$-2x_1 + 10x_2 \leq 29$

$x_1, x_2 \geq 0$

relaxed x_1, x_2 integers.

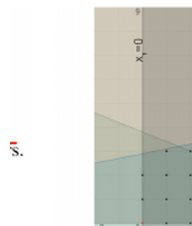
Optimal dictionary

ζ	=	$\frac{179}{3}$	$-\frac{7}{54}w_1$	$-\frac{73}{54}w_2$
x_1	=	$\frac{11}{3}$	$\frac{1}{54}w_1$	$-\frac{1}{54}w_2$
x_2	=	$\frac{7}{3}$	$\frac{1}{27}w_1$	$-\frac{5}{54}w_2$
w_3	=	13	$-\frac{5}{9}w_1$	$+\frac{8}{9}w_2$

Description

Slide 16 of 23 introduces the Gomory cuts method for solving integer programming problems. It presents an integer programming problem: maximize $8x_1 + 13x_2$ subject to three linear constraints and non-negativity constraints on x_1 and x_2 , where x_1 and x_2 are integers. The word "relaxed" is underlined, indicating that the integrality constraints on x_1 and x_2 are initially ignored, solving the linear programming relaxation. The slide then shows the optimal dictionary (simplex tableau) obtained from solving this relaxed linear program. The optimal solution to the relaxed problem is not integer. The figure shows a graphical representation of the feasible region of the integer program and the path taken by the simplex method applied to the LP relaxation. The red dot marks the optimal solution to the LP relaxation, which is not an integer solution. The context of linear programming is crucial because this slide demonstrates how the Gomory cuts method addresses the limitations of solving the linear programming relaxation of an integer program by adding cutting planes to eliminate non-integer solutions.

Figures



(a) A graph showing the feasible region of an integer program and the path taken by the simplex method.

Slide 17

Content

Gomory Cuts

Gomory Cuts

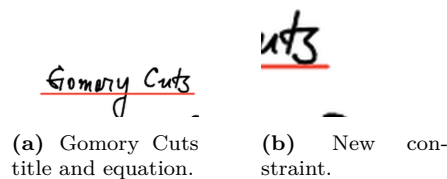
$$x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2 \implies x_1 + 0w_1 + 0w_2 \leq 3$$

new constraint

Description

Slide 17 of 23 demonstrates the application of Gomory cuts in integer programming. Building upon the previous slide, which presented the optimal solution to the linear programming relaxation of an integer program, this slide shows how to generate a Gomory cut. The equation $x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2$ represents a component of the optimal dictionary from the relaxed linear program (Slide 16). Since the optimal value of x_1 ($\frac{11}{3}$) is not an integer, a Gomory cut is derived. The process is not explicitly shown, but the result is a new constraint: $x_1 + 0w_1 + 0w_2 \leq 3$. This inequality is derived from the fractional part of x_1 's optimal value and eliminates the current non-integer solution while maintaining feasibility for all integer solutions. The new constraint is then added to the problem, and the process is repeated until an integer optimal solution is found. The underlined "Gomory Cuts" emphasizes the topic of the slide. The context of linear programming is crucial because this slide shows a key step in solving integer programming problems, which are a more complex extension of linear programming problems.

Figures



Slide 18

Content

Gomory Cuts

$$x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2 \implies x_1 + 0w_1 + 0w_2 \leq 3$$

new constraint

$$x_1 + \frac{5}{54}w_1 + \frac{5}{54}w_2 = \frac{11}{3}$$

round down

$$x_1 + 0w_1 + 0w_2 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2$$

$$\underline{\text{integer}} \implies \underline{\text{integer}} \leq \frac{11}{3}$$

$$\implies \text{integer} \leq \frac{11}{3}$$

Description

Slide 18 of 23 continues the explanation of Gomory cuts in integer programming. It starts with the equation $x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2$ from the optimal dictionary of the linear programming relaxation (Slide 16). The first line shows the derivation of a Gomory cut: $x_1 + 0w_1 + 0w_2 \leq 3$, which is obtained by considering the fractional part of x_1 's value and rounding down. The subsequent lines show the detailed steps involved in deriving this cut. The equation $x_1 + \frac{5}{54}w_1 + \frac{5}{54}w_2 = \frac{11}{3}$ is taken from the optimal simplex tableau. The fractional parts of the coefficients are dropped, resulting in $x_1 + 0w_1 + 0w_2 = \lfloor \frac{11}{3} \rfloor = 3$. The inequality $x_1 + 0w_1 + 0w_2 \leq 3$ is then obtained. The underlining emphasizes that the left-hand side is an integer and the right-hand side is an integer less than or equal to $\frac{11}{3}$. This new constraint cuts off the non-integer optimal solution from the previous slide while keeping all integer feasible solutions. The context of linear programming is crucial because this slide shows a detailed step-by-step derivation of a Gomory cut, a key technique for solving integer programming problems.

Figures

Gomory Cuts

$$x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2$$

$$x_1 + \frac{5}{54}w_1 + \frac{5}{54}w_2 = \frac{11}{3}$$

(a) Gomory Cuts
title and equations.

Slide 19

Content

Gomory Cuts

$$x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2 \implies x_1 + 0w_1 + 0w_2 \leq 3$$

new constraint

$$x_1 + \frac{5}{54}w_1 + \frac{5}{54}w_2 = \frac{11}{3}$$

round down

$$\frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2 \leq 3$$

$$-\frac{5}{54}w_1 - \frac{5}{54}w_2 \leq 3 - \frac{11}{3} = -\frac{2}{3}$$

$$\text{new slack var: } w_4 = -\frac{2}{3} + \frac{5}{54}w_1 + \frac{5}{54}w_2$$

Description

Slide 19 of 23 continues the explanation of Gomory cuts, focusing on adding a slack variable to the newly derived constraint. It begins with the equation $x_1 = \frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2$ from the optimal dictionary of the linear programming relaxation (Slide 16). A Gomory cut, $x_1 + 0w_1 + 0w_2 \leq 3$, was derived in the previous slide. This slide shows the detailed steps to obtain this inequality. The equation $x_1 + \frac{5}{54}w_1 + \frac{5}{54}w_2 = \frac{11}{3}$ is used. By rounding down the fractional parts, the inequality $\frac{11}{3} - \frac{5}{54}w_1 - \frac{5}{54}w_2 \leq 3$ is obtained. This is simplified to $-\frac{5}{54}w_1 - \frac{5}{54}w_2 \leq -\frac{2}{3}$. Finally, a slack variable w_4 is introduced to convert the inequality into an equation: $w_4 = -\frac{2}{3} + \frac{5}{54}w_1 + \frac{5}{54}w_2$. This new constraint, along with the slack variable, is added to the linear program's dictionary to continue the process of finding an integer optimal solution. The context of linear programming is crucial because this slide demonstrates a key step in the iterative process of the Gomory cut method, which is used to solve integer programming problems by adding cutting planes to the linear programming relaxation.

Figures

Gomory Cuts

$$x_1 = \frac{11}{3} - \frac{5}{54}x_2 - \frac{5}{54}w$$

$$x_1 + \frac{5}{54}x_2 + \frac{5}{54}w = \frac{11}{3}$$

(a) Gomory Cuts equations and derivations.

$$\frac{5}{4}w_2$$

$$= \frac{11}{4}$$

(b) New constraint and slack variable.

Slide 20

Content

Gomory Cuts

$$\zeta = \frac{179}{3} - \frac{7}{54}w_1 - \frac{73}{54}w_2$$

$$x_1 = \frac{3}{11} - \frac{54}{5}w_1 - \frac{54}{1}w_2$$

$$x_2 = \frac{7}{3} + \frac{27}{54}w_1 - \frac{8}{54}w_2$$

$$w_3 = 13 - \frac{54}{5}w_1 + \frac{9}{54}w_2$$

$$w_4 = -\frac{2}{3} + \frac{54}{5}w_1 + \frac{5}{54}w_2$$

new constraint

Opt. dict.

$$\zeta = \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2$$

$$x_1 = \frac{3}{5} + \frac{2}{5}w_4 - \frac{1}{10}w_2$$

$$x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2$$

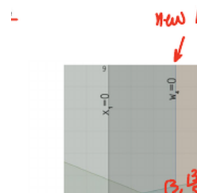
$$w_3 = 9 - 6w_4 + w_2$$

$$w_1 = \frac{36}{5} + \frac{54}{5}w_4 - \frac{1}{5}w_2$$

Description

Slide 20 of 23 illustrates the effect of adding a Gomory cut to an integer programming problem. The slide is divided into two main sections. The top section displays the initial simplex tableau before the Gomory cut is added. The tableau shows the objective function (ζ) and the values of the variables (x_1, x_2, w_3, w_4) expressed in terms of the non-basic variables (w_1, w_2). The variable w_4 represents the slack variable introduced in the previous slide. The red arrow points to the new constraint (w_4) generated by the Gomory cut method. The bottom section shows the optimal dictionary after the Gomory cut (w_4) has been added. The values of the variables are updated to reflect the new constraint. The key observation is that the fractional values of x_1 , x_2 , and w_3 from the initial tableau are now integers or close to integers in the updated tableau. A figure on the right shows a graphical representation of the feasible region before and after the addition of the Gomory cut. The new constraint creates a new boundary, cutting off a portion of the feasible region that contains non-integer solutions, while still containing all integer feasible solutions. The point $(3, \frac{11}{3})$ is highlighted, representing the optimal solution before the cut, which is not an integer solution. The context of linear programming is crucial because this slide demonstrates a key step in the iterative process of the Gomory cut method, which is used to solve integer programming problems by adding cutting planes to the linear programming relaxation. The figure visually confirms the effect of the Gomory cut on the feasible region.

Figures



(a) Figure 23.17. The integer program with our first new Gomory cut constraint added.

Gomory Cuts

$$z = \frac{120}{11} - \frac{7}{22} w_1 - \frac{23}{11} w_2$$

(b) Two simplex tableaus showing the before and after addition of a Gomory cut.

Slide 21

Content

Gomory Cuts

Opt. dict.

$$\zeta = \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2$$

$$x_1 = \frac{3}{5} + \frac{2}{5}w_4 - \frac{1}{10}w_2$$

$$x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2$$

$$w_3 = 9 - 6w_4 + w_2$$

$$w_1 = \frac{36}{5} + \frac{54}{5}w_4 - \frac{1}{5}w_2$$

$$x_2 - \frac{3}{5}w_4 + \frac{1}{10}w_2 = \frac{13}{5}$$

$$x_2 - w_4 = \frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2$$

$$\underline{\text{int.}} \quad \underline{\text{int.}} \leq \frac{13}{5} \implies \underline{\text{int.}} \leq 2$$

Description

Slide 21 of 23 demonstrates the derivation of a new Gomory cut from the optimal dictionary obtained after adding the previous Gomory cut. The top part shows the optimal dictionary, expressing the objective function (ζ) and variables (x_1, x_2, w_3, w_1) in terms of non-basic variables (w_4, w_2). The focus is on the equation for x_2 : $x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2$. The next lines show the derivation of a new constraint. The equation $x_2 - \frac{3}{5}w_4 + \frac{1}{10}w_2 = \frac{13}{5}$ is rewritten as $x_2 - w_4 = \frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2$. Since x_2 and w_4 are integers, the left-hand side is an integer. The fractional part of the right-hand side is dropped, leading to the inequality $\text{int.} \leq \frac{13}{5}$, which simplifies to $\text{int.} \leq 2$. This new constraint, $x_2 - w_4 \leq 2$, is a Gomory cut that will further restrict the feasible region and help to find an integer optimal solution. The underlining emphasizes that the left-hand side is an integer and the right-hand side is an integer less than or equal to $\frac{13}{5}$. The context of linear programming is crucial because this slide shows a detailed step-by-step derivation of another Gomory cut, a key technique for solving integer programming problems by iteratively adding cutting planes to the linear programming relaxation.

Figures

Gomory Cuts
Opt. dict.

$$\begin{aligned} \zeta &= \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2 \\ x_1 &= \frac{3}{5} + \frac{2}{5}w_4 - \frac{1}{10}w_2 \\ x_2 &= \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2 \\ \dots &= \dots \end{aligned}$$

(a) Optimal dictionary and derivation of a new constraint.

cuts

$$\begin{aligned} w_2 \\ w_2 \\ \dots \end{aligned}$$

(b) Derivation of a new constraint.

Slide 22

Content

Gomory Cuts

Opt. dict.

$$\zeta = \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2$$

$$x_1 = \frac{3}{5} + \frac{2}{5}w_4 - \frac{1}{10}w_2$$

$$x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2$$

$$w_3 = 9 - 6w_4 + w_2$$

$$w_1 = \frac{36}{5} + \frac{54}{5}w_4 - \frac{1}{5}w_2$$

$$x_2 - \frac{3}{5}w_4 + \frac{1}{10}w_2 = \frac{13}{5}$$

$$x_2 - w_4 = \frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2$$

$$\underline{\text{int.}} \quad \underline{\text{int.}} \leq \frac{13}{5} \implies \underline{\text{int.}} \leq 2$$

$$\frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2 \leq 2$$

$$\text{New slack: } w_5 = 2 - \left(\frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2 \right) = -\frac{3}{5} + \frac{3}{5}w_4 + \frac{1}{10}w_2$$

Description

Slide 22 of 23 continues the process of generating Gomory cuts to solve an integer programming problem. It starts with the optimal dictionary from the previous slide, highlighting the equation for x_2 : $x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2$. A new Gomory cut is derived by focusing on the fractional part of the right-hand side of this equation.

The equation is rearranged to $x_2 - w_4 = \frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2$. Since x_2 and w_4 are integers, the left-hand side is an integer. The inequality $\underline{\text{int.}} \leq \frac{13}{5}$ is obtained by dropping the fractional part of the right-hand side, which simplifies to $\underline{\text{int.}} \leq 2$. This inequality represents a new Gomory cut. A new slack variable, w_5 , is introduced to convert the inequality into an equation: $w_5 = 2 - \left(\frac{13}{5} - \frac{3}{5}w_4 - \frac{1}{10}w_2 \right) = -\frac{3}{5} + \frac{3}{5}w_4 + \frac{1}{10}w_2$. This new constraint, along with the slack variable w_5 , is added to the linear program's dictionary to continue the iterative process of finding an integer optimal solution. The underlining emphasizes that the left-hand side is an integer. The context of linear programming is crucial because this slide demonstrates another key step in the iterative process of the Gomory cut method, which is used to solve integer programming problems by adding cutting planes to the linear programming relaxation.

Figures

Gomory Cuts
Opt. dict.

$$\begin{array}{l} \zeta = \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2 \\ x_1 = \frac{3}{5} - \frac{2}{5}w_4 - \frac{1}{10}w_2 \\ x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2 \\ \dots = 0 \end{array}$$

(a) Optimal dictionary and derivation of a new constraint.

$$\begin{array}{l} w_3 = 9 - 6w_4 + w_2 \\ w_1 = \frac{36}{5} + \frac{54}{5}w_4 - \frac{1}{5}w_2 \end{array}$$

$$x_2 - \frac{3}{5}w_4 + \frac{1}{10}w_2 =$$

(b) Derivation of a new constraint and slack variable.

Slide 23

Content

Gomory Cuts

$$\zeta = \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2$$

$$x_1 = \frac{3}{5} + \frac{2}{5}w_4 - \frac{1}{10}w_2$$

$$x_2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2$$

$$w_3 = 9 - 6w_4 + w_2$$

$$w_1 = \frac{36}{5} + \frac{54}{5}w_4 - \frac{1}{5}w_2$$

$$w_5 = -\frac{3}{5} + \frac{1}{5}w_4 + \frac{1}{10}w_2$$

new slack (int.)

Opt. soln (int.)

$$\zeta = 55 - \frac{14}{5}w_5 - \frac{52}{10}w_2$$

$$x_1 = 2 - \frac{2}{5}w_5 + \frac{1}{10}w_2$$

$$x_2 = 3 - \frac{3}{5}w_5 - \frac{1}{10}w_2$$

$$w_3 = 3 - 10w_5 + 2w_2$$

$$w_1 = 18 + 18w_5 - 2w_2$$

$$w_4 = 1 + \frac{5}{3}w_5 - \frac{1}{6}w_2$$

Description

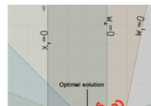
Slide 23 of 23 presents the final solution to an integer programming problem using Gomory cuts. The slide is divided into three main sections. The top section shows the initial simplex tableau before the second Gomory cut is added. The middle section displays the optimal dictionary after adding the second Gomory cut. Notice that all variables now have integer values, indicating that an integer optimal solution has been found. The bottom section shows a graphical representation of the feasible region. The feasible region is progressively reduced by adding two Gomory cuts, ultimately leading to an integer optimal solution at the point (2,3). The red arrow points to the optimal integer solution (2,3). The text "Opt. soln (int.)" indicates that this is the optimal integer solution. The context of linear programming is crucial because this slide demonstrates the successful application of the Gomory cut method to solve an integer programming problem. The iterative addition of Gomory cuts progressively reduces the feasible region until an integer optimal solution is found. The figure visually confirms the effect of the Gomory cuts on the feasible region, showing how they cut off non-integer solutions while preserving the integer solutions.

Figures

Gomory Cuts

$$\begin{array}{l} \zeta = \frac{289}{5} - \frac{14}{5}w_4 - \frac{13}{10}w_2 \\ 1 = \frac{3}{5} - \frac{2}{5}w_4 + \frac{1}{10}w_2 \\ 2 = \frac{13}{5} + \frac{3}{5}w_4 - \frac{1}{10}w_2 \\ 3 = 9 - 6w_4 + w_2 \\ 4 = \frac{36}{5} + \frac{54}{5}w_4 - \frac{1}{5}w_2 \end{array}$$

(a) Initial simplex tableau before adding the second Gomory cut.



(b) Optimal simplex tableau after adding the second Gomory cut.

$$\begin{array}{l} \zeta = 55 - \frac{14}{5}w_5 - \frac{52}{10}w_2 \\ 1 = 2 - \frac{2}{5}w_5 + \frac{1}{10}w_2 \\ 2 = 3 - \frac{3}{5}w_5 - \frac{1}{10}w_2 \\ 3 = 3 - 10w_5 + 2w_2 \\ 4 = 18 + 18w_5 - 2w_2 \\ 5 = 1 + \frac{5}{3}w_5 - \frac{1}{6}w_2 \end{array}$$

(c) Graphical representation of the feasible region with two Gomory cuts.